

PyTorch Fundamentals

IIT Madras BS in Data Science & Electronic Systems

Course: Deep Learning & Generative AI • Week 1 & 2

Notebook Overview & Key Topics Covered:

Tensors, Tensor Attributes, Mathematical Operations, Reshaping, Slicing, Matrix Multiplication, Device Agnostic Execution & CUDA Acceleration

1 PyTorch Fundamentals: A Comprehensive Introduction to Tensor Operations and Deep Learning Foundations

Course: [Course Name/Code]

Institution: IIT Madras

Author: Prof. Ganapathy Krishnamurthi

Date: [Date]

1.1 Learning Objectives

By the end of this tutorial, students will be able to:

1. **Understand** the fundamental concepts of tensors and their role in deep learning
2. **Create** and manipulate tensors using PyTorch's tensor operations
3. **Apply** tensor operations for mathematical computations in deep learning contexts
4. **Implement** tensor reshaping, indexing, and aggregation operations
5. **Utilize** GPU acceleration for tensor computations
6. **Debug** common tensor-related errors in PyTorch applications

1.2 Prerequisites

- Basic understanding of Python programming
- Familiarity with NumPy arrays (recommended)
- Linear algebra fundamentals (vectors, matrices, matrix multiplication)
- Basic machine learning concepts (recommended but not required)

1.3 1. Overview and Theoretical Background

1.3.1 1.1 Introduction to PyTorch

PyTorch is an open-source machine learning framework developed by Facebook's AI Research lab (FAIR). It provides a Python-based scientific computing package that serves as:

1. **A replacement for NumPy** with the power of Graphics Processing Units (GPUs)
2. **A deep learning research platform** that provides maximum flexibility and speed

1.3.2 1.2 Applications and Use Cases

PyTorch enables researchers and practitioners to:

- **Data Manipulation:** Process and transform multidimensional data structures efficiently
- **Algorithm Development:** Implement machine learning and deep learning algorithms using automatic differentiation
- **Model Prototyping:** Rapidly develop and test neural network architectures
- **Production Deployment:** Scale models from research to production environments

1.3.3 1.3 Industry Adoption

PyTorch has gained significant adoption across various sectors:

- **Technology Companies:** Meta (Facebook), Tesla, Microsoft, and others utilize PyTorch for production systems
- **Research Institutions:** OpenAI and academic institutions leverage PyTorch for cutting-edge research
- **Academic Community:** As of 2022, PyTorch is the most utilized deep learning framework in academic publications according to Papers With Code

1.3.4 1.4 Advantages of PyTorch

Research-Friendly Design: - Dynamic computational graphs allow for flexible model architectures - Pythonic interface that integrates seamlessly with the Python ecosystem - Extensive debugging capabilities with standard Python debugging tools

Performance Optimization: - Efficient GPU utilization through CUDA integration - Optimized tensor operations for numerical computations - Support for distributed training across multiple devices

1.3.5 1.5 Theoretical Foundation: Understanding Tensors

Tensors represent the fundamental data structure in machine learning and deep learning applications. They serve as generalized mathematical objects that can represent:

- **Scalars** (0-dimensional tensors): Single numerical values
- **Vectors** (1-dimensional tensors): Arrays of numbers with directional properties
- **Matrices** (2-dimensional tensors): Rectangular arrays for linear transformations
- **Higher-order tensors** (n-dimensional): Multi-dimensional data structures for complex data

1.3.6 1.6 Curriculum Structure

This tutorial systematically covers the essential tensor operations required for deep learning applications:

Module	Learning Outcomes	Practical Applications
2. Tensor Introduction	Understand tensor mathematics and data representation	Foundation for all ML/DL operations
3. Tensor Creation	Master various tensor initialization methods	Data preprocessing and model initialization
4. Tensor Inspection	Extract metadata and properties from tensors	Debugging and model analysis
5. Tensor Operations	Perform mathematical operations on tensors	Forward and backward propagation in neural networks
6. Shape Manipulation	Reshape and reorganize tensor dimensions	Data preprocessing and model compatibility
7. Tensor Indexing	Access and modify specific tensor elements	Data sampling and feature extraction
8. NumPy Integration	Convert between PyTorch tensors and NumPy arrays	Interoperability with existing data science workflows
9. Reproducibility	Control randomness for consistent results	Ensuring reproducible research and debugging
10. GPU Acceleration	Leverage GPU computing for faster operations	Scaling computations for large datasets and models

1.3.7 1.7 Mathematical Notation

Throughout this tutorial, we will use the following mathematical conventions:

- **Scalars:** Lowercase italic letters (e.g., x , y , α)
- **Vectors:** Lowercase bold letters (e.g., $\mathbf{x}$, $\mathbf{v}$)
- **Matrices:** Uppercase bold letters (e.g., $\mathbf{X}$, $\mathbf{W}$)
- **Tensors:** Uppercase bold letters with tensor rank notation (e.g., $\mathcal{X} \in \mathbb{R}^{d_1 \times d_2 \times \dots \times d_n}$)

1.4 2. Environment Setup and Library Imports

1.4.1 2.1 Installation Requirements

Local Installation Prerequisites: Before executing code in this tutorial locally, ensure PyTorch is properly installed following the official PyTorch installation guide. The installation process varies based on your operating system, Python version, and desired CUDA support.

Cloud Environment: For **Google Colab** users, PyTorch and associated libraries are pre-installed and ready for use.

1.4.2 2.2 Compatibility Notes

- **Python Version:** Python 3.7 or higher recommended
- **CUDA Support:** Optional but recommended for GPU acceleration
- **Memory Requirements:** Minimum 4GB RAM for basic operations
- **Dependencies:** NumPy, Matplotlib (for visualization examples)

1.4.3 2.3 Library Import and Version Verification

We begin by importing the PyTorch library and verifying the installed version. This step ensures compatibility and helps with debugging potential version-specific issues.

```
# Import the PyTorch library
# PyTorch is the primary library for tensor operations and deep learning
import torch

# Display the PyTorch version
# This is crucial for ensuring compatibility with code examples
# Different PyTorch versions may have slight API differences
print(f"PyTorch Version: {torch.__version__}")
```

PYTHON IN [8]

OUTPUT

PyTorch Version: 2.4.1+cu121

1.5 3. Introduction to Tensors: Mathematical Foundations

1.5.1 3.1 Theoretical Background

Tensors constitute the fundamental building blocks of machine learning computations. In the context of deep learning, tensors serve as the primary data structure for representing:

- **Input data:** Images, text sequences, audio signals, and tabular data
- **Model parameters:** Weights and biases in neural networks
- **Intermediate computations:** Activations and gradients during training
- **Output predictions:** Classification probabilities and regression values

1.5.2 3.2 Mathematical Definition

A tensor **T** of rank **n** (or order **n**) is a mathematical object with **n** indices, where each index can range over a specific dimension. Formally:

$$\mathbf{T} \in \mathbb{R}^{d_1 \times d_2 \times \dots \times d_n}$$

Where: - **$d_1, d_2, \dots, d_n$** represent the size of each dimension - **n** is the rank/order of the tensor - $\mathbb{R}$ denotes the set of real numbers (though tensors can contain other data types)

1.5.3 3.3 Computational Significance

Tensors enable efficient computation through:

1. **Vectorization:** Operations on entire arrays rather than individual elements
2. **Parallelization:** GPU acceleration for simultaneous computations
3. **Automatic Differentiation:** Gradient computation for optimization algorithms
4. **Memory Efficiency:** Optimized storage and access patterns

1.5.4 3.4 Data Representation Through Tensors

Tensors provide a unified mathematical framework for representing diverse data types in numerical form. This numerical representation is essential because machine learning algorithms operate exclusively on numerical data.

1.5.4.1 3.4.1 Image Representation Example

Consider a color image representation as a 3-dimensional tensor with shape [channels, height, width]:

- **Channels (C):** Color information (typically 3 for RGB: Red, Green, Blue)
- **Height (H):** Vertical resolution in pixels
- **Width (W):** Horizontal resolution in pixels

For example, a standard image might have shape [3, 224, 224], representing: - **3 color channels** (RGB) - **224 pixels** in height - **224 pixels** in width - **Total elements:** $3 \times 224 \times 224 = 150,528$ numerical values

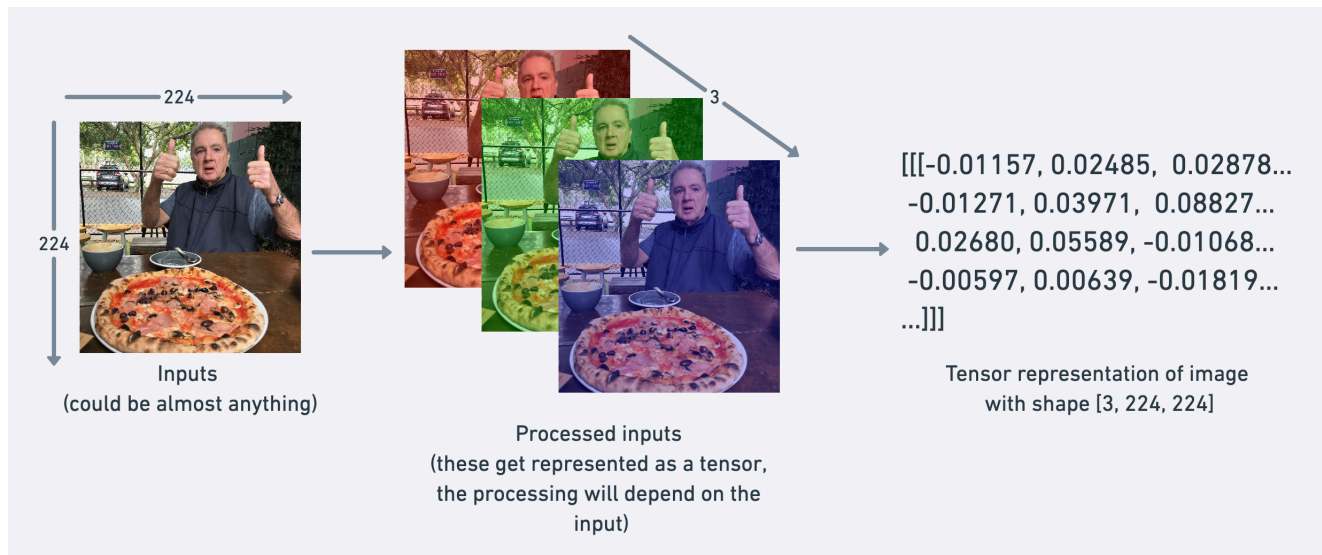


Figure 1: Tensor representation of an image showing the breakdown into color channels and spatial dimensions

Figure 3.1: Transformation of visual data into tensor representation for computational processing. Source: learnpytorch.io

1.5.4.2 3.4.2 Data Type Implications

The choice of tensor dimensions and data organization significantly impacts: - **Memory consumption:** Higher dimensions require more storage - **Computational complexity:** More dimensions increase operation costs - **Model architecture design:** Network layers must match tensor shapes - **Training efficiency:** Optimal batch sizes depend on tensor dimensions

1.6 4. Tensor Creation and Initialization Methods

1.6.1 4.1 Theoretical Framework

The `torch.Tensor` class serves as the fundamental data structure in PyTorch. Understanding tensor creation methods is crucial for:

- **Data preprocessing:** Converting raw data into tensor format
- **Model initialization:** Creating parameter tensors with appropriate shapes
- **Experimentation:** Generating synthetic data for testing and validation

1.6.2 4.2 Tensor Hierarchy and Classification

4.2.1 Scalar Tensors (Rank 0)

A **scalar** represents a single numerical value and constitutes a **zero-dimensional tensor**. In mathematical notation: - **Mathematical representation:** $s \in \mathbb{R}$ - **PyTorch shape:** `torch.Size([])` - **Dimensions:** 0

Properties: - Contains exactly one element - No directional information - Often used for loss values, learning rates, and single metrics

```
# Create a scalar tensor (0-dimensional tensor)
# Scalars contain a single numerical value and have no dimensions
# They are fundamental building blocks for more complex tensor operations
```

PYTHON IN [14]

```

scalar = torch.tensor(7) # Create a scalar tensor with value 7

# Display the scalar tensor
print(f"Scalar tensor: {scalar}")
print(f"Scalar value: {scalar.item()}") # Extract the Python number
print(f"Data type: {scalar.dtype}")      # Check the data type
print(f"Shape: {scalar.shape}")          # Shape is empty for scalars
print(f"Size: {scalar.size()}")         # Alternative way to check shape

# The scalar tensor object contains metadata beyond just the value:
# - dtype: The data type (default is inferred from input)
# - device: The computational device (CPU or GPU)
# - requires_grad: Whether to track gradients for automatic differentiation

scalar

```

OUTPUT

```

Scalar tensor: 7
Scalar value: 7
Data type: torch.int64
Shape: torch.Size([])
Size: torch.Size([])

```

OUTPUT

```

tensor(7)

```

1.6.3 4.3 Tensor Dimensionality Analysis

Tensor dimensionality (or rank) indicates the number of indices required to specify an element within the tensor. The `ndim` attribute provides this crucial information for tensor manipulation and debugging.

PYTHON IN [16]

```

# Check the number of dimensions (rank) of the scalar tensor
# For a scalar, this should return 0 since it has no dimensions
dimensionality = scalar.ndim

print(f"Number of dimensions (rank): {dimensionality}")
print(f"Verification: A scalar has {dimensionality} dimensions")

# Understanding dimensionality is crucial for:
# 1. Matrix operations (shapes must be compatible)
# 2. Neural network layer design (input/output dimensions must match)
# 3. Data preprocessing (ensuring correct tensor shapes)

dimensionality

```

OUTPUT

```

Number of dimensions (rank): 0
Verification: A scalar has 0 dimensions

```

OUTPUT

```

0

```

1.6.4 4.4 Value Extraction from Tensors

To retrieve the underlying numerical value from a tensor, PyTorch provides the `item()` method. This method is essential for:

- **Metric reporting:** Extracting loss values for logging
- **Conditional operations:** Using tensor values in Python control structures
- **Debugging:** Inspecting individual tensor elements

Important constraint: The `item()` method only works with tensors containing exactly one element (single-element tensors).

PYTHON IN [18]

```
# Extract the Python numerical value from a single-element tensor
# The item() method converts a PyTorch tensor to a Python scalar
# This is essential when you need to use tensor values in standard Python operations

python_value = scalar.item()

print(f"Original tensor: {scalar}")
print(f"Extracted Python value: {python_value}")
print(f"Type of tensor: {type(scalar)}")
print(f"Type of extracted value: {type(python_value)}")

# Practical applications of item():
# 1. Loss monitoring: loss_value = loss_tensor.item()
# 2. Metric computation: accuracy = correct_predictions.item() / total_samples
# 3. Conditional logic: if accuracy_tensor.item() > threshold: ...
# 4. Logging: print(f"Epoch {epoch}, Loss: {loss.item():.4f}")

# Note: Attempting item() on multi-element tensors will raise an error
# For multi-element tensors, use indexing to access specific elements first

python_value
```

OUTPUT

```
Original tensor: 7
Extracted Python value: 7
Type of tensor: <class 'torch.Tensor'>
Type of extracted value: <class 'int'>
```

OUTPUT

7

1.6.5 4.5 Vector Tensors (Rank 1)

1.6.5.1 4.5.1 Mathematical Definition

A **vector** represents a one-dimensional tensor containing multiple numerical values arranged in a specific order. In mathematical notation: - **Mathematical representation:** $\mathbf{v} \in \mathbb{R}^n$ - **PyTorch shape:** `torch.Size([n])`

- **Dimensions:** 1

1.6.5.2 4.5.2 Properties and Characteristics

Vectors possess several important properties: - **Ordered sequence:** Elements have specific positions (indices) - **Directional information:** Can represent direction and magnitude - **Flexible representation:** Can encode various types of information

Real-world applications: - **Housing features:** [bedrooms, bathrooms, square_feet] - **Word embeddings:** Dense representations of words in NLP - **Feature vectors:** Extracted features from raw data - **Probability distributions:** Categorical probability outputs

PYTHON IN [20]

```
# Create a vector tensor (1-dimensional tensor)
# Vectors contain multiple values arranged in a single dimension
# They are fundamental for representing sequences, features, and embeddings

vector = torch.tensor([7.0, 7.0]) # Create a vector with two identical elements

print(f"Vector tensor: {vector}")
print(f"Data type: {vector.dtype}")
print(f"Dimension: {vector.dim()}")
print(f"Shape: {vector.shape}")
print(f"Number of elements: {vector.numel()}") # Total number of elements

# Vector interpretation examples:
# [7, 7] could represent:
# - Coordinates in 2D space (x=7, y=7)
```

```
# - Two identical measurements
# - A repeated pattern or signal
# - Feature values for two attributes

# Mathematical properties:
print(f"L2 norm (Euclidean length): {torch.norm(vector).item():.4f}")
print(f"Sum of elements: {torch.sum(vector).item()}")
print(f"Mean of elements: {torch.mean(vector.float()).item():.4f}")

vector
```

OUTPUT

```
Vector tensor: tensor([7., 7.])
Data type: torch.float32
Dimension: 1
Shape: torch.Size([2])
Number of elements: 2
L2 norm (Euclidean length): 9.8995
Sum of elements: 14.0
Mean of elements: 7.0000
```

OUTPUT

```
tensor([7., 7.])
```

1.6.6 4.6 Matrix Tensors (Rank 2)

Mathematical Definition: A **matrix** represents a two-dimensional tensor organized in rows and columns. In mathematical notation: - **Mathematical representation:** $\mathbf{M} \in \mathbb{R}^{m \times n}$ - **PyTorch shape:** `torch.Size([m, n])` - **Dimensions:** 2

Properties: - **Rows (m):** First dimension, representing horizontal sequences - **Columns (n):** Second dimension, representing vertical sequences

- **Linear transformations:** Matrices can represent linear mappings between vector spaces - **Data organization:** Natural structure for tabular data and 2D relationships

PYTHON IN [22]

```
# Create a matrix tensor (2-dimensional tensor)
# Matrices are fundamental for linear algebra operations in machine learning
# They represent linear transformations, weight matrices, and 2D data structures

MATRIX = torch.tensor([[7.0, 8.0],      # First row: [7, 8]
                        [9.0, 10.0]])   # Second row: [9, 10]

print(f"Matrix tensor:\n{MATRIX}")
print(f"Data type: {MATRIX.dtype}")
print(f"Dimension: {MATRIX.dim()}") # Should return 2 for a matrix
print(f"Shape: {MATRIX.shape}") # Returns torch.Size([rows, columns])
print(f"Number of dimensions: {MATRIX.ndim}")
print(f"Total elements: {MATRIX.numel()}")

# Matrix properties and operations:
print(f"\nMatrix properties:")
print(f"Determinant: {torch.det(MATRIX.float()).item():.4f}") # Requires float type
print(f"Trace (sum of diagonal): {torch.trace(MATRIX).item()}")
print(f"Frobenius norm: {torch.norm(MATRIX).item():.4f}")

# Matrix interpretations:
# - Weight matrix in neural networks (input_features × output_features)
# - Transformation matrix for geometric operations
# - Correlation matrix for feature relationships
# - Adjacency matrix for graph representations

MATRIX
```

OUTPUT

```
Matrix tensor:
tensor([[ 7.,  8.],
        [ 9., 10.]])
Data type: torch.float32
Dimension: 2
Shape: torch.Size([2, 2])
Number of dimensions: 2
Total elements: 4

Matrix properties:
Determinant: -2.0000
Trace (sum of diagonal): 17.0
Frobenius norm: 17.1464
```

```
OUTPUT
tensor([[ 7.,  8.],
        [ 9., 10.]])
```

1.6.7 4.7 Higher-Order Tensors (Rank ≥ 3)

Mathematical Definition: Higher-order tensors extend beyond matrices to represent multi-dimensional data structures. For a rank-3 tensor: - **Mathematical representation:** $\mathcal{T} \in \mathbb{R}^{d_1 \times d_2 \times d_3}$ - **PyTorch shape:** `torch.Size([d1, d2, d3])` - **Dimensions:** 3 or higher

Applications in Deep Learning: - **Batch processing:** [batch_size, features] Or [batch_size, channels, height, width] - **Sequence modeling:** [batch_size, sequence_length, feature_dimension] - **Computer vision:** [batch_size, channels, height, width] for image batches - **Video processing:** [batch_size, time_steps, channels, height, width]

PYTHON IN [24]

```
# Create a higher-order tensor (3-dimensional tensor)
# This example creates a rank-3 tensor with shape [1, 3, 3]
# The triple bracket notation [[[...]]] indicates three dimensions

TENSOR = torch.tensor([[[1, 2, 3],      # First 3x3 matrix, row 1
                        [3, 6, 9],      # First 3x3 matrix, row 2
                        [2, 4, 5]]])    # First 3x3 matrix, row 3

print(f"3D Tensor:\n{TENSOR}")
print(f"Data type: {TENSOR.dtype}")
print(f"Dimension: {TENSOR.dim()}") # Should return 3 for a 3D tensor
print(f"Shape: {TENSOR.shape}") # torch.Size([1, 3, 3])
print(f"Number of dimensions: {TENSOR.ndim}")
print(f"Total elements: {TENSOR.numel()}")

# Shape interpretation: [1, 3, 3]
# - Dimension 0: 1 "slice" or "channel" (outermost dimension)
# - Dimension 1: 3 rows within each slice
# - Dimension 2: 3 columns within each row

print(f"\nDimensional analysis:")
print(f"Outer dimension (axis 0): {TENSOR.shape[0]} - Number of 3x3 matrices")
print(f"Middle dimension (axis 1): {TENSOR.shape[1]} - Number of rows per matrix")
print(f"Inner dimension (axis 2): {TENSOR.shape[2]} - Number of columns per row")

# Real-world analogy: This could represent:
# - A single grayscale image patch (1 channel, 3x3 pixels)
# - One time step of a 3x3 feature map in a CNN
# - A small kernel/filter in convolutional operations
# - A single sample in a batch of 3x3 matrices

TENSOR
```

```
OUTPUT
3D Tensor:
tensor([[[1, 2, 3],
         [3, 6, 9],
```

```
[2, 4, 5]])
Data type: torch.int64
Dimension: 3
Shape: torch.Size([1, 3, 3])
Number of dimensions: 3
Total elements: 9

Dimensional analysis:
Outer dimension (axis 0): 1 - Number of 3x3 matrices
Middle dimension (axis 1): 3 - Number of rows per matrix
Inner dimension (axis 2): 3 - Number of columns per row
```

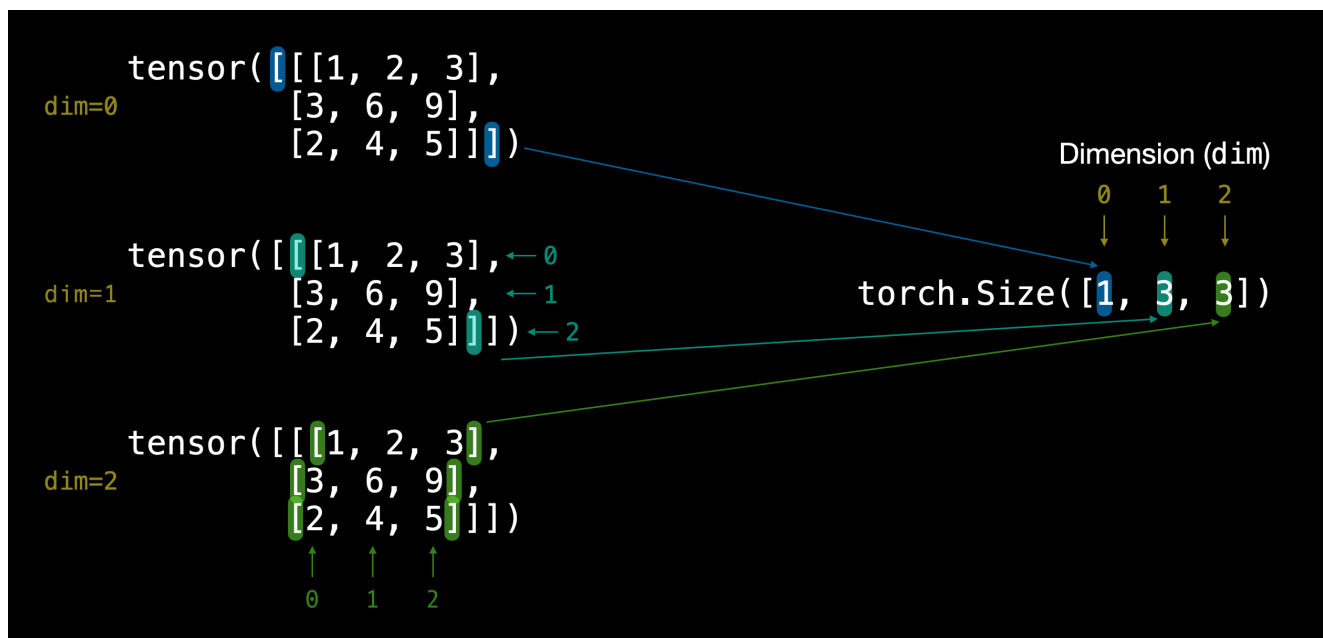
OUTPUT

```
tensor([[[1, 2, 3],
         [3, 6, 9],
         [2, 4, 5]]])
```

Alright, it outputs `torch.Size([1, 3, 3])`.

The dimensions go outer to inner.

That means there's 1 dimension of 3 by 3.



Source: learnpytorch.io

The one we just created could be the sales numbers for a steak and almond butter store.

	A	B	C	D	E	F	G	H	I
1	Day of week	1	2	3					
2	Steak sales	3	6	9					
3	Almond butter sales	2	4	5					
4									
5									
6									
7									

Figure 2: a simple tensor in google sheets showing day of week, steak sales and almond butter sales

Source: learnpytorch.io

Note: You might've noticed the usage of lowercase letters for scalar and vector and uppercase letters for MATRIX and TENSOR. This was on purpose. In practice, you'll often see scalars and vectors denoted as lowercase letters such as y or a . And matrices and tensors denoted as uppercase letters such as X or W .

You also might notice the names matrix and tensor used interchangeably. This is common. Since in PyTorch you're often dealing with `torch.Tensors` (hence the tensor name), however, the shape and dimensions of what's inside will dictate what it actually is.

1.6.8 Summary

Name	What is it?	Number of dimensions	Lower or upper (usually/example)
scalar	a single number	0	Lower (a)
vector	a number with direction (e.g. wind speed with direction) but can also have many other numbers	1	Lower (y)
matrix	a 2-dimensional array of numbers	2	Upper (Q)
tensor	an n -dimensional array of numbers	can be any number, a 0-dimension tensor is a scalar, a 1-dimension tensor is a vector	Upper (x)

1.7 5. Specialized Tensor Initialization Methods

1.7.1 5.1 Zero and One Initialization

Theoretical Background: Zero and one tensors serve crucial roles in deep learning applications:

1.7.1.1 5.1.1 Zero Tensors

- **Mathematical representation:** $\mathbf{0} \in \mathbb{R}^{d_1 \times d_2 \times \dots \times d_n}$ where all elements equal 0
- **Applications:**

- **Bias initialization:** Starting with zero bias in neural networks
- **Padding operations:** Adding zero-valued boundaries to tensors
- **Masking:** Creating attention masks or sequence padding
- **Memory allocation:** Pre-allocating tensors before filling with computed values

1.7.1.2 5.1.2 One Tensors

- **Mathematical representation:** $\mathbf{1} \in \mathbb{R}^{d_1 \times d_2 \times \dots \times d_n}$ where all elements equal 1
- **Applications:**
 - **Identity operations:** Creating identity matrices for linear algebra
 - **Normalization:** Serving as multiplicative identity elements
 - **Template creation:** Base tensors for subsequent operations
 - **Testing:** Simplified computations for debugging and validation

PYTHON IN [31]

```
# Create a tensor filled with zeros using torch.zeros()
# The size parameter determines the tensor dimensions
# This is essential for initializing tensors before computation

zeros = torch.zeros(size=(3, 4)) # Create a 3x4 matrix of zeros

print(f"Zero tensor:\n{zeros}")
print(f"Shape: {zeros.shape}")
print(f>Data type: {zeros.dtype}") # Default is torch.float32
print(f"Device: {zeros.device}") # Default is CPU
print(f"Total elements: {zeros.numel()}")

# Memory and computational considerations:
print(f"\nTensor properties:")
print(f"Memory usage (bytes): {zeros.element_size() * zeros.numel()}")
print(f"Is contiguous in memory: {zeros.is_contiguous()}")
print(f"Requires gradient: {zeros.requires_grad}")

# Common use cases for zero tensors:
# 1. Neural network bias initialization: bias = torch.zeros(output_features)
# 2. Attention masks: mask = torch.zeros(seq_len, seq_len)
# 3. Gradient accumulation: grad_accumulator = torch.zeros_like(parameter)
# 4. Batch processing placeholders: batch_data = torch.zeros(batch_size, features)

# Mathematical verification:
print(f"\nMathematical properties:")
print(f"Sum of all elements: {torch.sum(zeros).item()}")
print(f"Mean of all elements: {torch.mean(zeros).item()}")
print(f"Standard deviation: {torch.std(zeros).item()}")

zeros, zeros.dtype
```

```
OUTPUT
Zero tensor:
tensor([[0., 0., 0., 0.],
        [0., 0., 0., 0.],
        [0., 0., 0., 0.]])
Shape: torch.Size([3, 4])
Data type: torch.float32
Device: cpu
Total elements: 12

Tensor properties:
Memory usage (bytes): 48
Is contiguous in memory: True
Requires gradient: False

Mathematical properties:
Sum of all elements: 0.0
Mean of all elements: 0.0
Standard deviation: 0.0
```

OUTPUT

```
(tensor([[0., 0., 0., 0.],
        [0., 0., 0., 0.],
        [0., 0., 0., 0.]]),
 torch.float32)
```

PYTHON IN [32]

```
# Create a tensor filled with ones using torch.ones()
# Similar to zeros, but all elements are initialized to 1.0
# Useful for multiplicative identity operations and normalization

ones = torch.ones(size=(3, 4)) # Create a 3x4 matrix of ones

print(f"Ones tensor:\n{ones}")
print(f"Shape: {ones.shape}")
print(f"Data type: {ones.dtype}") # Default is torch.float32
print(f"Device: {ones.device}")

# Mathematical properties of ones tensor:
print(f"\nMathematical analysis:")
print(f"Sum of all elements: {torch.sum(ones).item()}") # Should equal total elements
print(f"Product of all elements: {torch.prod(ones).item()}") # Should equal 1.0
print(f"Mean of all elements: {torch.mean(ones).item()}") # Should equal 1.0
print(f"L2 norm: {torch.norm(ones).item():.4f}") # Square root of number of elements

# Practical applications:
# 1. Identity matrix creation: I = torch.eye(n) or torch.ones(n,n) with modifications
# 2. Masking operations: valid_mask = torch.ones(sequence_length)
# 3. Weight initialization scaling: weights = torch.ones(shape) * init_scale
# 4. Attention mechanisms: attention_weights = torch.ones(num_heads, seq_len, seq_len)

# Computational efficiency note:
# Both zeros() and ones() are optimized operations that don't require
# element-by-element assignment, making them very fast for large tensors

ones, ones.dtype
```

OUTPUT

```
Ones tensor:
tensor([[1., 1., 1., 1.],
        [1., 1., 1., 1.],
        [1., 1., 1., 1.]])
Shape: torch.Size([3, 4])
Data type: torch.float32
Device: cpu

Mathematical analysis:
Sum of all elements: 12.0
Product of all elements: 1.0
Mean of all elements: 1.0
L2 norm: 3.4641
```

OUTPUT

```
(tensor([[1., 1., 1., 1.],
        [1., 1., 1., 1.],
        [1., 1., 1., 1.]])
 torch.float32)
```

1.7.2 5.2 Random Tensor Generation

Theoretical Foundation: Random tensors are fundamental in machine learning for:

1. **Parameter Initialization:** Breaking symmetry in neural networks
2. **Stochastic Processes:** Modeling uncertainty and variability
3. **Data Augmentation:** Creating synthetic training examples
4. **Monte Carlo Methods:** Approximating complex probability distributions

1.7.2.1 5.2.1 Uniform Random Distribution

The `torch.rand()` function generates tensors with values sampled from a uniform distribution over the interval $[0, 1)$:

Mathematical representation: $X \sim U(0,1)$ - **Probability density function:** $f(x) = 1$ for $x \in [0,1)$, 0 otherwise - **Mean:** $\mu = 0.5$ - **Variance:** $\sigma^2 = 1/12 \approx 0.083$

PYTHON IN [34]

```
# Create a tensor with random values from uniform distribution [0, 1)
# Random tensors are crucial for neural network weight initialization
# They help break symmetry and enable effective learning

random_tensor = torch.rand(size=(3, 4)) # 3x4 matrix with random values

print(f"Random tensor:\n{random_tensor}")
print(f"Shape: {random_tensor.shape}")
print(f"Data type: {random_tensor.dtype}")

# Statistical analysis of the random tensor:
print(f"\nStatistical properties:")
print(f"Minimum value: {torch.min(random_tensor).item():.6f}")
print(f"Maximum value: {torch.max(random_tensor).item():.6f}")
print(f"Mean value: {torch.mean(random_tensor).item():.6f}") # Should be ~0.5
print(f"Standard deviation: {torch.std(random_tensor).item():.6f}") # Should be ~0.289

# Theoretical vs. empirical comparison:
# For uniform distribution U(0,1): theoretical_mean = 0.5, theoretical_std = 1/sqrt(12) ≈ 0.289
theoretical_std = 1.0 / (12**0.5)
print(f"Theoretical std for U(0,1): {theoretical_std:.6f}")

# Applications in deep learning:
# 1. Weight initialization: weights = torch.rand(input_size, output_size) * scale
# 2. Dropout simulation: dropout_mask = (torch.rand(size) > dropout_rate).float()
# 3. Data augmentation: noise = torch.rand(data.shape) * noise_level
# 4. Monte Carlo sampling: samples = torch.rand(num_samples, dimensions)

# Note: Each execution will produce different values due to randomness
# This is essential for stochastic training processes in machine learning

random_tensor, random_tensor.dtype
```

OUTPUT

```
Random tensor:
tensor([[0.5607, 0.0444, 0.4183, 0.7015],
        [0.4447, 0.5234, 0.1832, 0.2898],
        [0.5075, 0.8066, 0.1623, 0.8025]])
Shape: torch.Size([3, 4])
Data type: torch.float32

Statistical properties:
Minimum value: 0.044396
Maximum value: 0.806584
Mean value: 0.453747
Standard deviation: 0.248735
Theoretical std for U(0,1): 0.288675
```

OUTPUT

```
(tensor([[0.5607, 0.0444, 0.4183, 0.7015],
        [0.4447, 0.5234, 0.1832, 0.2898],
        [0.5075, 0.8066, 0.1623, 0.8025]]),
 torch.float32)
```

PYTHON IN [35]

```
# Create a random tensor with image-like dimensions
# Common computer vision tensor shape: (height, width, channels)
# This simulates a standard 224x224 RGB image used in many CV models
```

```

random_image_size_tensor = torch.rand(size=(224, 224, 3))

print(f"Image tensor shape: {random_image_size_tensor.shape}")
print(f"Number of dimensions: {random_image_size_tensor.ndim}")
print(f"Total elements: {random_image_size_tensor.numel():,}") # Format with commas

# Memory analysis for large tensors:
element_size = random_image_size_tensor.element_size() # Bytes per element
total_memory = random_image_size_tensor.numel() * element_size
print(f"\nMemory analysis:")
print(f"Bytes per element: {element_size}")
print(f"Total memory usage: {total_memory:,} bytes ({total_memory/1024/1024:.2f} MB)")

# Dimension interpretation for computer vision:
height, width, channels = random_image_size_tensor.shape
print(f"\nImage properties:")
print(f"Height: {height} pixels")
print(f"Width: {width} pixels")
print(f"Channels: {channels} (RGB color channels)")
print(f"Total pixels: {height * width:,}")

# Standard image formats in deep learning:
# - ImageNet standard: 224x224x3 (this example)
# - CIFAR-10: 32x32x3
# - MNIST: 28x28x1 (grayscale)
# - High-resolution: 512x512x3 or larger

# Note: In practice, we often use (C, H, W) format for PyTorch models
# This tensor uses (H, W, C) format, which is common for data loading

random_image_size_tensor.shape, random_image_size_tensor.ndim

```

OUTPUT

```

Image tensor shape: torch.Size([224, 224, 3])
Number of dimensions: 3
Total elements: 150,528

```

Memory analysis:

```

Bytes per element: 4
Total memory usage: 602,112 bytes (0.57 MB)

```

Image properties:

```

Height: 224 pixels
Width: 224 pixels
Channels: 3 (RGB color channels)
Total pixels: 50,176

```

OUTPUT

```

(torch.Size([224, 224, 3]), 3)

```

1.7.3 5.3 Creating a range and tensors like

Sometimes you might want a range of numbers, such as 1 to 10 or 0 to 100.

```
torch.arange(start, end, step)
```

Where: * start = start of range (e.g. 0) * end = end of range (e.g. 10) * step = how many steps in between each value (e.g. 1)

```

# Create a range of values 0 to 10
zero_to_ten = torch.arange(start=0, end=10, step=1)
zero_to_ten

```

PYTHON IN [37]

OUTPUT

```
tensor([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

Sometimes you might want one tensor of a certain type with the same shape as another tensor.

For example, a tensor of all zeros with the same shape as a previous tensor.

To do so you can use `torch.zeros_like(input)` or `torch.ones_like(input)` which return a tensor filled with zeros or ones in the same shape as the input respectively.

PYTHON IN [39]

```
# Can also create a tensor of zeros similar to another tensor
ten_zeros = torch.zeros_like(input=zero_to_ten) # will have same shape
ten_zeros
```

OUTPUT

```
tensor([0, 0, 0, 0, 0, 0, 0, 0, 0, 0])
```

1.7.4 5.4 Tensor datatypes

There are many different tensor datatypes available in PyTorch.

Generally if you see `torch.cuda` anywhere, the tensor is being used for GPU (since Nvidia GPUs use a computing toolkit called CUDA).

The most common type (and generally the default) is `torch.float32` or `torch.float`. But there's also 16-bit floating point (`torch.float16` or `torch.half`) and 64-bit floating point (`torch.float64` or `torch.double`). There's also 8-bit, 16-bit, 32-bit and 64-bit integers.

Note: An integer is a flat round number like 7 whereas a float has a decimal 7.0.

Resources: * See the PyTorch documentation for a list of all available tensor datatypes. * Read the Wikipedia page for an overview of what is precision in computing.

Let's see how to create some tensors with specific datatypes. We can do so using the `dtype` parameter.

PYTHON IN [42]

```
# Default datatype for tensors is float32
float_32_tensor = torch.tensor([3.0, 6.0, 9.0],
                                dtype=None, # defaults to None, which is torch.float32 or whatever datatype
                                is passed
                                device=None, # defaults to None, which uses the default tensor type
                                requires_grad=False) # if True, operations performed on the tensor are
recorded

float_32_tensor.shape, float_32_tensor.dtype, float_32_tensor.device
```

OUTPUT

```
(torch.Size([3]), torch.float32, device(type='cpu'))
```

1.8 6 Manipulating tensors

In deep learning, data (images, text, video, audio, protein structures, etc) gets represented as tensors.

A model learns by investigating those tensors and performing a series of operations on tensors to create a representation of the patterns in the input data.

These operations are often a wonderful dance between: * Addition * Subtraction * Multiplication (element-wise) * Division * Matrix multiplication

1.8.1 6.1 Basic operations

Let's start with a few of the fundamental operations, addition (+), subtraction (-), multiplication (*).

PYTHON IN [46]

```
# Create a tensor of values and add a number to it
tensor = torch.tensor([1, 2, 3])
tensor + 10
```

```
OUTPUT
tensor([11, 12, 13])
```

```
# Multiply it by 10
tensor * 10
```

PYTHON IN [47]

```
OUTPUT
tensor([10, 20, 30])
```

Let's subtract a number and this time we'll reassign the tensor variable.

```
# Subtract and reassign
tensor = tensor - 10
tensor
```

PYTHON IN [49]

```
OUTPUT
tensor([-9, -8, -7])
```

```
# Add and reassign
tensor = tensor + 10
tensor
```

PYTHON IN [50]

```
OUTPUT
tensor([1, 2, 3])
```

PyTorch also has a bunch of built-in functions like `torch.mul()` (short for multiplication) and `torch.add()` to perform basic operations.

```
# Can also use torch functions
torch.multiply(tensor, 10)
```

PYTHON IN [52]

```
OUTPUT
tensor([10, 20, 30])
```

```
# Element-wise multiplication (each element multiplies its equivalent, index 0->0, 1->1, 2->2)
print(tensor, "*", tensor)
print("Equals:", tensor * tensor)
```

PYTHON IN [53]

```
OUTPUT
tensor([1, 2, 3]) * tensor([1, 2, 3])
Equals: tensor([1, 4, 9])
```

1.8.2 6.2 Matrix Multiplication: The Foundation of Neural Networks

Theoretical Significance: Matrix multiplication represents the most fundamental operation in deep learning. The famous phrase “*Attention is All You Need*” from the Transformer paper could equally apply to matrix multiplication in neural networks.

1.8.2.1 6.2.1 Mathematical Definition

For matrices $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{B} \in \mathbb{R}^{n \times p}$, their product $\mathbf{C} = \mathbf{AB} \in \mathbb{R}^{m \times p}$ is defined as:

$$C[i,j] = \sum_{k=1}^n A[i,k] \times B[k,j]$$

1.8.2.2 6.2.2 Deep Learning Applications

Neural Network Forward Pass: - Linear layers: $\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$ - Attention mechanisms: $\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}(\mathbf{Q}\mathbf{K}^T / \sqrt{d})\mathbf{V}$ - Convolutional operations: Implemented as matrix multiplications via `im2col`

Training Process: - Gradient computation: $\partial \mathcal{L} / \partial \mathbf{W}$ involves matrix products - Backpropagation: Chain rule applications through matrix operations

1.8.2.3 6.2.3 Matrix Multiplication Rules and Implementation

PyTorch implements matrix multiplication through the `torch.matmul()` function, which handles various tensor dimensions automatically.

Critical Rules for Matrix Multiplication:

Rule 1: Inner Dimension Compatibility The inner dimensions of the matrices must match for multiplication to be possible: - **Valid:** $(m, n) @ (n, p) \rightarrow (m, p)$ ✓ - **Invalid:** $(m, n) @ (k, p)$ where $n \neq k$ ✗

Examples: - $(3, 2) @ (3, 2) \rightarrow$ **Invalid** (inner dimensions: $2 \neq 3$) - $(2, 3) @ (3, 2) \rightarrow$ **Valid** (inner dimensions: $3 = 3$) → Result: $(2, 2)$ - $(3, 2) @ (2, 3) \rightarrow$ **Valid** (inner dimensions: $2 = 2$) → Result: $(3, 3)$

Rule 2: Output Shape Determination The resulting matrix has the shape of the outer dimensions: - $(m, \mathbf{n}) @ (\mathbf{n}, p) \rightarrow (m, p)$ - The inner dimensions ($\mathbf{n}$) are “consumed” during multiplication

Operator Notation: - **Recommended:** `torch.matmul(A, B)` OR `A @ B` - **Alternative:** `torch.mm(A, B)` for 2D matrices only - **Note:** The `@` operator is the standard Python matrix multiplication symbol (PEP 465)

Computational Complexity: - **Time complexity:** $O(mnp)$ for $(m,n) @ (n,p)$ - **Space complexity:** $O(mp)$ for the result matrix - **GPU acceleration:** Highly optimized on modern GPUs using libraries like cuBLAS

```
import torch
tensor = torch.tensor([1, 2, 3])
tensor.shape
```

PYTHON IN [56]

```
OUTPUT
torch.Size([3])
```

The difference between element-wise multiplication and matrix multiplication is the addition of values.

For our tensor variable with values `[1, 2, 3]`:

Operation	Calculation	Code
Element-wise multiplication	$[1*1, 2*2, 3*3] = [1, 4, 9]$	<code>tensor * tensor</code>
Matrix multiplication	$[1*1 + 2*2 + 3*3] = [14]$	<code>tensor.matmul(tensor)</code>

```
# Element-wise matrix multiplication
tensor * tensor
```

PYTHON IN [58]

```
OUTPUT
tensor([1, 4, 9])
```

```
# Demonstrate matrix multiplication using torch.matmul()
# This example shows the mathematical difference between element-wise and matrix multiplication

# Using the previously defined tensor: [1, 2, 3]
print(f"Vector tensor: {tensor}")
print(f"Vector shape: {tensor.shape}")

# Matrix multiplication for vectors computes the dot product
# Mathematical formula:  $\mathbf{v} \cdot \mathbf{v} = \sum_i \mathbf{v}_i \times \mathbf{v}_i = \mathbf{v}_1^2 + \mathbf{v}_2^2 + \mathbf{v}_3^2$ 
```

PYTHON IN [59]

```

matrix_result = torch.matmul(tensor, tensor)

print(f"\n=== MATRIX MULTIPLICATION (DOT PRODUCT) ===")
print(f"Operation: torch.matmul({tensor}, {tensor})")
print(f"Mathematical computation: (1×1) + (2×2) + (3×3) = 1 + 4 + 9 = 14")
print(f"Result: {matrix_result}")
print(f"Result shape: {matrix_result.shape}") # Scalar result (0-dimensional)
print(f"Result type: {type(matrix_result.item())} value = {matrix_result.item()}")

# Interpretation in machine learning contexts:
print(f"\n=== MACHINE LEARNING INTERPRETATIONS ===")
print("1. Similarity measure: Higher dot product indicates more similar vectors")
print("2. Energy/norm calculation: ||v||² = v·v (when v=tensor)")
print(f"3. Vector magnitude: ||v|| = √(v·v) = √{matrix_result.item()} = {torch.sqrt(matrix_result).item():.4f}")
print("4. Neural network computation: Linear layer output = input·weights")

# Computational efficiency note:
print(f"\n=== COMPUTATIONAL NOTES ===")
print("Matrix multiplication is highly optimized in PyTorch:")
print("- Uses BLAS (Basic Linear Algebra Subprograms) libraries")
print("- Automatically leverages multiple CPU cores")
print("- GPU acceleration available via cuBLAS on CUDA devices")
print("- Essential operation for neural network forward/backward passes")

matrix_result

```

OUTPUT

```

Vector tensor: tensor([1, 2, 3])
Vector shape: torch.Size([3])

```

```

=== MATRIX MULTIPLICATION (DOT PRODUCT) ===
Operation: torch.matmul(tensor([1, 2, 3]), tensor([1, 2, 3]))
Mathematical computation: (1×1) + (2×2) + (3×3) = 1 + 4 + 9 = 14
Result: 14
Result shape: torch.Size([])
Result type: <class 'int'> value = 14

=== MACHINE LEARNING INTERPRETATIONS ===
1. Similarity measure: Higher dot product indicates more similar vectors
2. Energy/norm calculation: ||v||² = v·v (when v=tensor)
3. Vector magnitude: ||v|| = √(v·v) = √14 = 3.7417
4. Neural network computation: Linear layer output = input·weights

=== COMPUTATIONAL NOTES ===
Matrix multiplication is highly optimized in PyTorch:
- Uses BLAS (Basic Linear Algebra Subprograms) libraries
- Automatically leverages multiple CPU cores
- GPU acceleration available via cuBLAS on CUDA devices
- Essential operation for neural network forward/backward passes

```

OUTPUT

```

tensor(14)

```

PYTHON IN [60]

```

# Can also use the "@" symbol for matrix multiplication, though not recommended
tensor @ tensor

```

OUTPUT

```

tensor(14)

```

PYTHON IN [62]

1.8.3 6.3 Aggregation

```
# Create a tensor
x = torch.arange(0, 100, 10)
x
```

OUTPUT

```
tensor([ 0, 10, 20, 30, 40, 50, 60, 70, 80, 90])
```

```
print(f"Minimum: {x.min()}")
print(f"Maximum: {x.max()}")
print(f"Sum: {x.sum()}")
```

PYTHON IN [63]

OUTPUT

```
Minimum: 0
Maximum: 90
Sum: 450
```

```
print(f"Mean: {x.mean()}") # this will give error
```

PYTHON IN [64]

```
print(f"Mean: {x.type(torch.float32).mean()}") # won't work without float datatype
```

PYTHON IN [65]

OUTPUT

```
Mean: 45.0
```

Note: You may find some methods such as `torch.mean()` require tensors to be in `torch.float32` (the most common) or another specific datatype, otherwise the operation will fail.

You can also do the same as above with `torch` methods.

```
torch.max(x), torch.min(x), torch.mean(x.type(torch.float32)), torch.sum(x)
```

PYTHON IN [67]

OUTPUT

```
(tensor(90), tensor(0), tensor(45.), tensor(450))
```

1.8.4 6.4 Positional min/max

You can also find the index of a tensor where the max or minimum occurs with `torch.argmax()` and `torch.argmin()` respectively.

This is helpful incase you just want the position where the highest (or lowest) value is and not the actual value itself

```
# Create a tensor
tensor = torch.arange(10, 100, 10)
print(f"Tensor: {tensor}")

# Returns index of max and min values
print(f"Index where max value occurs: {tensor.argmax()}")
print(f"Index where min value occurs: {tensor.argmin()}")
```

PYTHON IN [69]

OUTPUT

```
Tensor: tensor([10, 20, 30, 40, 50, 60, 70, 80, 90])
Index where max value occurs: 8
Index where min value occurs: 0
```

1.8.5 6.5 Reshaping, Stacking, Squeezing and Unsqueezing

Often times you'll want to reshape or change the dimensions of your tensors without actually changing the values inside them.

Method	One-line description
<code>torch.reshape(input, shape)</code>	Reshapes input to shape (if compatible), can also use <code>torch.Tensor.reshape()</code> .
<code>Tensor.view(shape)</code>	Returns a view of the original tensor in a different shape but shares the same data as the original tensor.
<code>torch.stack(tensors, dim=0)</code>	Concatenates a sequence of tensors along a new dimension (dim), all tensors must be same size.
<code>torch.squeeze(input)</code>	Squeezes input to remove all the dimensions with value 1.
<code>torch.unsqueeze(input, dim)</code>	Returns input with a dimension value of 1 added at dim.
<code>torch.permute(input, dims)</code>	Returns a view of the original input with its dimensions permuted (rearranged) to dims.

Why do any of these?

Because deep learning models (neural networks) are all about manipulating tensors in some way. And because of the rules of matrix multiplication, if you've got shape mismatches, you'll run into errors. These methods help you make sure the right elements of your tensors are mixing with the right elements of other tensors.

```
# Create a tensor
import torch
x = torch.arange(1., 8.)
x, x.shape
```

PYTHON IN [72]

```
OUTPUT
(tensor([1., 2., 3., 4., 5., 6., 7.]), torch.Size([7]))
```

Now let's add an extra dimension with `torch.reshape()`.

```
# Add an extra dimension
x_resaped = x.reshape(1, 7)
x_resaped, x_resaped.shape
```

PYTHON IN [74]

```
OUTPUT
(tensor([[1., 2., 3., 4., 5., 6., 7.]]), torch.Size([1, 7]))
```

```
# Change view (keeps same data as original but changes view)
# See more: https://stackoverflow.com/a/54507446/7900723
z = x.view(1, 7)
z, z.shape
```

PYTHON IN [75]

```
OUTPUT
(tensor([[1., 2., 3., 4., 5., 6., 7.]]), torch.Size([1, 7]))
```

Remember though, changing the view of a tensor with `torch.view()` really only creates a new view of the *same* tensor. So changing the view changes the original tensor too.

```
# Changing z changes x
z[:, 0] = 5
z, x
```

PYTHON IN [77]

OUTPUT

```
(tensor([[5., 2., 3., 4., 5., 6., 7.]]), tensor([5., 2., 3., 4., 5., 6., 7.]))
```

If we wanted to stack our new tensor on top of itself five times, we could do so with `torch.stack()`.

PYTHON IN [79]

```
# Stack tensors on top of each other
x_stacked = torch.stack([x, x, x, x], dim=0) # try changing dim to dim=1 and see what happens
x_stacked
```

OUTPUT

```
tensor([[5., 2., 3., 4., 5., 6., 7.],
        [5., 2., 3., 4., 5., 6., 7.],
        [5., 2., 3., 4., 5., 6., 7.],
        [5., 2., 3., 4., 5., 6., 7.]])
```

How about removing all single dimensions from a tensor?

To do so you can use `torch.squeeze()`

PYTHON IN [81]

```
print(f"Previous tensor: {x_resaped}")
print(f"Previous shape: {x_resaped.shape}")

# Remove extra dimension from x_resaped
x_squeezed = x_resaped.squeeze()
print(f"\nNew tensor: {x_squeezed}")
print(f"New shape: {x_squeezed.shape}")
```

OUTPUT

```
Previous tensor: tensor([[5., 2., 3., 4., 5., 6., 7.]])
Previous shape: torch.Size([1, 7])

New tensor: tensor([5., 2., 3., 4., 5., 6., 7.])
New shape: torch.Size([7])
```

And to do the reverse of `torch.squeeze()` you can use `torch.unsqueeze()` to add a dimension value of 1 at a specific index.

PYTHON IN [83]

```
print(f"Previous tensor: {x_squeezed}")
print(f"Previous shape: {x_squeezed.shape}")

## Add an extra dimension with unsqueeze
x_unsqueezed = x_squeezed.unsqueeze(dim=0)
print(f"\nNew tensor: {x_unsqueezed}")
print(f"New shape: {x_unsqueezed.shape}")
```

OUTPUT

```
Previous tensor: tensor([5., 2., 3., 4., 5., 6., 7.])
Previous shape: torch.Size([7])

New tensor: tensor([[5., 2., 3., 4., 5., 6., 7.]])
New shape: torch.Size([1, 7])
```

You can also rearrange the order of axes values with `torch.permute(input, dims)`, where the input gets turned into a view with new dims.

PYTHON IN [85]

```
# Create tensor with specific shape
x_original = torch.rand(size=(224, 224, 3))

# Permute the original tensor to rearrange the axis order
```

```
x_permuted = x_original.permute(2, 0, 1) # shifts axis 0->1, 1->2, 2->0

print(f"Previous shape: {x_original.shape}")
print(f"New shape: {x_permuted.shape}")
```

OUTPUT

```
Previous shape: torch.Size([224, 224, 3])
New shape: torch.Size([3, 224, 224])
```

Note: Because permuting returns a *view* (shares the same data as the original), the values in the permuted tensor will be the same as the original tensor and if you change the values in the view, it will change the values of the original.

1.8.6 6.6 Indexing

```
# Create a tensor
import torch
x = torch.arange(1, 10).reshape(1, 3, 3)
x, x.shape
```

PYTHON IN [88]

OUTPUT

```
(tensor([[[[1, 2, 3],
          [4, 5, 6],
          [7, 8, 9]]]],
        torch.Size([1, 3, 3]))
```

Indexing values goes outer dimension -> inner dimension (check out the square brackets).

```
# Let's index bracket by bracket
print(f"First square bracket:\n{x[0]}")
print(f"Second square bracket: {x[0][0]}")
print(f"Third square bracket: {x[0][0][0]}")
```

PYTHON IN [90]

OUTPUT

```
First square bracket:
tensor([[1, 2, 3],
        [4, 5, 6],
        [7, 8, 9]])
Second square bracket: tensor([1, 2, 3])
Third square bracket: 1
```

You can also use `:` to specify “all values in this dimension” and then use a comma `(,)` to add another dimension.

```
# Get all values of 0th dimension and the 0 index of 1st dimension
x[:, 0]
```

PYTHON IN [92]

OUTPUT

```
tensor([[1, 2, 3]])
```

```
# Get all values of 0th & 1st dimensions but only index 1 of 2nd dimension
x[:, :, 1]
```

PYTHON IN [93]

OUTPUT

```
tensor([[2, 5, 8]])
```

```
# Get all values of the 0 dimension but only the 1 index value of the 1st and 2nd dimension
x[:, 1, 1]
```

PYTHON IN [94]

```
OUTPUT
tensor([5])
```

```
# Get index 0 of 0th and 1st dimension and all values of 2nd dimension
x[0, 0, :] # same as x[0][0]
```

PYTHON IN [95]

```
OUTPUT
tensor([1, 2, 3])
```

1.9 7 PyTorch tensors & NumPy

PyTorch has functionality to interact with numpy.

The two main methods you'll want to use for NumPy to PyTorch (and back again) are: * `torch.from_numpy(ndarray)` - NumPy array -> PyTorch tensor. * `torch.Tensor.numpy()` - PyTorch tensor -> NumPy array.

```
# NumPy array to tensor
import torch
import numpy as np
array = np.arange(1.0, 8.0)
tensor = torch.from_numpy(array)
array, tensor
```

PYTHON IN [98]

```
OUTPUT
(array([1., 2., 3., 4., 5., 6., 7.]),
 tensor([1., 2., 3., 4., 5., 6., 7.], dtype=torch.float64))
```

Note: By default, NumPy arrays are created with the datatype `float64` and if you convert it to a PyTorch tensor, it'll keep the same datatype (as above).

However, many PyTorch calculations default to using `float32`.

So if you want to convert your NumPy array (`float64`) -> PyTorch tensor (`float64`) -> PyTorch tensor (`float32`), you can use `tensor = torch.from_numpy(array).type(torch.float32)`.

```
# Tensor to NumPy array
tensor = torch.ones(7) # create a tensor of ones with dtype=float32
numpy_tensor = tensor.numpy() # will be dtype=float32 unless changed
tensor, numpy_tensor
```

PYTHON IN [100]

```
OUTPUT
(tensor([1., 1., 1., 1., 1., 1., 1.]),
 array([1., 1., 1., 1., 1., 1., 1.], dtype=float32))
```

1.10 8 Reproducibility

Everytime the below code block runs, `random_tensor_A` gets a new value.

```
import torch

# Create two random tensors
```

PYTHON IN [103]

```
random_tensor_A = torch.rand(3, 4)

print(f"Tensor A:\n{random_tensor_A}\n")
```

OUTPUT

```
Tensor A:
tensor([[0.7539, 0.1952, 0.0050, 0.3068],
        [0.1165, 0.9103, 0.6440, 0.7071],
        [0.6581, 0.4913, 0.8913, 0.1447]])
```

PYTHON IN [104]

```
# Set the random seed
RANDOM_SEED=42 # try changing this to different values and see what happens to the numbers below

# Feed the random seed to PyTorch
torch.manual_seed(seed=RANDOM_SEED)

random_tensor_A = torch.rand(3, 4)
print(f"Tensor A:\n{random_tensor_A}\n")
```

OUTPUT

```
Tensor A:
tensor([[0.8823, 0.9150, 0.3829, 0.9593],
        [0.3904, 0.6009, 0.2566, 0.7936],
        [0.9408, 0.1332, 0.9346, 0.5936]])
```

PYTHON IN [105]

```
# Set the random seed
RANDOM_SEED=42 # try changing this to different values and see what happens to the numbers below

# Feed the random seed to GPU
torch.cuda.manual_seed(seed=RANDOM_SEED)

random_tensor_A = torch.rand(3, 4, device='cuda')
print(f"Tensor A:\n{random_tensor_A}\n")
```

OUTPUT

```
Tensor A:
tensor([[0.6130, 0.0101, 0.3984, 0.0403],
        [0.1563, 0.4825, 0.7362, 0.4060],
        [0.5189, 0.2867, 0.2416, 0.9228]], device='cuda:0')
```

1.11 9 GPU Acceleration for High-Performance Computing

1.11.1 9.1 Theoretical Foundation

Graphics Processing Units (GPUs) have revolutionized deep learning by providing massively parallel computational capabilities. Understanding GPU utilization is essential for practical deep learning applications.

1.11.1.1 9.1.1 CPU vs GPU Architecture

Central Processing Unit (CPU): - **Design philosophy:** Optimized for sequential processing and complex control logic - **Core count:** Typically 4-32 cores with sophisticated caching - **Memory:** Large, hierarchical cache systems - **Best for:** Complex branching, single-threaded performance, system management

Graphics Processing Unit (GPU): - **Design philosophy:** Optimized for parallel processing of simple operations - **Core count:** Thousands of simple cores (e.g., 2,048-10,496 CUDA cores) - **Memory:** High-bandwidth memory (HBM) with lower latency tolerance - **Best for:** Matrix operations, element-wise computations, data parallelism

1.11.1.2 9.1.2 CUDA Ecosystem

CUDA (Compute Unified Device Architecture) enables general-purpose computing on NVIDIA GPUs: - **Programming model:** Parallel computing platform and API - **Memory hierarchy:** Global, shared, constant, and texture memory

types - **Execution model:** Kernel launches with thread blocks and grids - **Library ecosystem:** cuBLAS, cuDNN, cuSPARSE for optimized operations

Note: This tutorial focuses on NVIDIA GPUs with CUDA support. Alternative platforms include AMD ROCm and Intel oneAPI, but CUDA remains the most widely supported in deep learning frameworks.

1.11.2 9.2 Getting a GPU

To check if you've got access to a Nvidia GPU, you can run `!nvidia-smi` where the `!` (also called bang) means “run this on the command line”.

```
!nvidia-smi
```

PYTHON IN [109]

OUTPUT

Thu Aug 21 17:54:19 2025

```
+-----+
| NVIDIA-SMI 535.183.01                  Driver Version: 535.183.01   CUDA Version: 12.2   |
+-----+-----+
| GPU  Name                Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap|      Memory-Usage | GPU-Util  Compute M. |
|=====+=====+
|  0  Quadro K2200                Off | 00000000:01:00.0 On  |         N/A         |
| 43%   46C    P8              1W / 39W | 423MiB / 4096MiB |    21%    Default   |
|                                     |                      | N/A              |
+-----+-----+
|  1  NVIDIA GeForce RTX 3090        Off | 00000000:04:00.0 Off |         N/A         |
| 30%   37C    P8             25W / 350W | 358MiB / 24576MiB |     0%    Default   |
|                                     |                      | N/A              |
+-----+-----+

+-----+
| Processes:                                |
| GPU   GI    CI          PID    Type    Process name                        GPU Memory |
|   ID   ID           |                     |           | Usage |
|=====+=====+
|  0   N/A   N/A         1297     G   /usr/lib/xorg/Xorg                  N/A      |
... [Output truncated for brevity] ...
```

If you don't have a Nvidia GPU accessible, the above will output something like:

NVIDIA-SMI has failed because it couldn't communicate with the NVIDIA driver. Make sure that the latest NVIDIA driver is installed and running.

If you do have a GPU, the line above will output something like:

Wed Jan 19 22:09:08 2022

```
+-----+
| NVIDIA-SMI 495.46                  Driver Version: 460.32.03   CUDA Version: 11.2   |
+-----+-----+
| GPU  Name                Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap|      Memory-Usage | GPU-Util  Compute M. |
|=====+=====+
|  0  Tesla P100-PCIE...        Off | 00000000:00:04.0 Off |         0         |
| N/A   35C    P0             27W / 250W |   0MiB / 16280MiB |     0%    Default   |
|                                     |                      | N/A              |
+-----+-----+

+-----+
| Processes:                                |
| GPU   GI    CI          PID    Type    Process name                        GPU Memory |
|   ID   ID           |                     |           | Usage |
|=====+=====+
| No running processes found              |
+-----+-----+
```

1.11.3 9.3 Getting PyTorch to run on the GPU

You can test if PyTorch has access to a GPU using `torch.cuda.is_available()`.

PYTHON IN [112]

```
# Check CUDA availability and system configuration
# This diagnostic is crucial for ensuring optimal performance in deep learning workflows

import torch

# Primary CUDA availability check
cuda_available = torch.cuda.is_available()

print("=== GPU AVAILABILITY DIAGNOSTIC ===")
print(f"CUDA Available: {cuda_available}")

if cuda_available:
    # Detailed GPU information for performance optimization
    print(f"\n=== GPU HARDWARE DETAILS ===")
    print(f"Number of GPUs: {torch.cuda.device_count()}")
    print(f"Current GPU Device: {torch.cuda.current_device()}")
    print(f"GPU Name: {torch.cuda.get_device_name(0)}")

    # Memory analysis - critical for batch size optimization
    print(f"\n=== MEMORY CONFIGURATION ===")
    memory_allocated = torch.cuda.memory_allocated(0)
    memory_reserved = torch.cuda.memory_reserved(0)
    total_memory = torch.cuda.get_device_properties(0).total_memory

    print(f"Total GPU Memory: {total_memory / 1024**3:.2f} GB")
    print(f"Currently Allocated: {memory_allocated / 1024**2:.2f} MB")
    print(f"Currently Reserved: {memory_reserved / 1024**2:.2f} MB")
    print(f"Available Memory: {(total_memory - memory_reserved) / 1024**3:.2f} GB")

    # CUDA version compatibility
    print(f"\n=== SOFTWARE VERSIONS ===")
    print(f"PyTorch Version: {torch.__version__}")
    print(f"CUDA Version: {torch.version.cuda}")
    print(f"cuDNN Version: {torch.backends.cudnn.version()}")
    print(f"cuDNN Enabled: {torch.backends.cudnn.enabled}")

else:
    print("\n=== CPU-ONLY CONFIGURATION ===")
    print("GPU acceleration not available. Training will use CPU.")
    print("For large models, consider:")
    print("1. Cloud services (Google Colab, AWS, Azure)")
    print("2. CUDA-compatible GPU installation")
    print("3. Reduced model size and batch size for CPU training")

# Performance implications:
print(f"\n=== PERFORMANCE EXPECTATIONS ===")
if cuda_available:
    print("✓ GPU acceleration available - expect 10-100x speedup for large models")
    print("✓ Large batch sizes supported (limited by GPU memory)")
    print("✓ Suitable for production-scale training")
else:
    print("⚠ CPU-only mode - expect slower training")
    print("⚠ Smaller batch sizes recommended")
    print("⚠ Consider GPU resources for larger experiments")

cuda_available
```

OUTPUT

```
=== GPU AVAILABILITY DIAGNOSTIC ===
CUDA Available: True

=== GPU HARDWARE DETAILS ===
Number of GPUs: 2
```

```

Current GPU Device: 0
GPU Name: NVIDIA GeForce RTX 3090

=== MEMORY CONFIGURATION ===
Total GPU Memory: 23.69 GB
Currently Allocated: 0.00 MB
Currently Reserved: 0.00 MB
Available Memory: 23.69 GB

=== SOFTWARE VERSIONS ===
PyTorch Version: 2.4.1+cu121
CUDA Version: 12.1
cuDNN Version: 90100
cuDNN Enabled: True

=== PERFORMANCE EXPECTATIONS ===
✓ GPU acceleration available - expect 10-100x speedup for large models
✓ Large batch sizes supported (limited by GPU memory)
✓ Suitable for production-scale training

```

OUTPUT
True

```

# Set device type
device = "cuda" if torch.cuda.is_available() else "cpu"
device

```

PYTHON IN [113]

OUTPUT
'cuda'

If the above output "cuda" it means we can set all of our PyTorch code to use the available CUDA device (a GPU) and if the output is "cpu", our PyTorch code will stick with the CPU.

Note: In PyTorch, it's best practice to write **device agnostic code**. This means that the code will run on CPU (always available) or GPU (if available).

```

# Count number of devices
torch.cuda.device_count()

```

PYTHON IN [115]

OUTPUT
2

1.11.4 9.4 Putting tensors (and models) on the GPU

You can put tensors (and models, we'll see this later) on a specific device by calling `to(device)` on them. Where `device` is the target device you'd like the tensor (or model) to go to.

Why do this?

GPUs offer far faster numerical computing than CPUs do and if a GPU isn't available, because of our **device agnostic code** (see above), it'll run on the CPU.

Note: Putting a tensor on GPU using `to(device)` (e.g. `some_tensor.to(device)`) returns a copy of that tensor, e.g. the same tensor will be on CPU and GPU. To overwrite tensors, reassign them:

```
some_tensor = some_tensor.to(device)
```

Let's try creating a tensor and putting it on the GPU (if it's available).

```
# Create tensor (default on CPU)
tensor = torch.tensor([1, 2, 3])

# Tensor not on GPU
print(tensor, tensor.device)

# Move tensor to GPU (if available)
tensor_on_gpu = tensor.to(device)
tensor_on_gpu
```

PYTHON IN [117]

```
OUTPUT
tensor([1, 2, 3]) cpu
```

```
OUTPUT
tensor([1, 2, 3], device='cuda:0')
```

If you have a GPU available, the above code will output something like:

```
tensor([1, 2, 3]) cpu
tensor([1, 2, 3], device='cuda:0')
```

Notice the second tensor has `device='cuda:0'`, this means it's stored on the 0th GPU available (GPUs are 0 indexed, if two GPUs were available, they'd be `'cuda:0'` and `'cuda:1'` respectively, up to `'cuda:n'`).

1.11.5 9.5 Moving tensors back to the CPU

What if we wanted to move the tensor back to CPU?

For example, you'll want to do this if you want to interact with your tensors with NumPy (NumPy does not leverage the GPU).

Let's try using the `torch.Tensor.numpy()` method on our `tensor_on_gpu`.

```
# If tensor is on GPU, can't transform it to NumPy (this will error)
tensor_on_gpu.numpy()
```

PYTHON IN [120]

Instead, to get a tensor back to CPU and usable with NumPy we can use `Tensor.cpu()`.

```
# Instead, copy the tensor back to cpu
tensor_back_on_cpu = tensor_on_gpu.cpu().numpy()
tensor_back_on_cpu
```

PYTHON IN [122]

```
OUTPUT
array([1, 2, 3])
```

The above returns a copy of the GPU tensor in CPU memory so the original tensor is still on GPU.

```
tensor_on_gpu
```

PYTHON IN [124]

```
OUTPUT
tensor([1, 2, 3], device='cuda:0')
```

```
tensor_on_gpu.to('cpu')
```

PYTHON IN [125]

```
OUTPUT
tensor([1, 2, 3])
```

1.12 10 Most Common Errors

Because much of deep learning is multiplying and performing operations on matrices and matrices have a strict rule about what shapes and sizes can be combined, one of the most common errors you'll run into in deep learning is shape mismatches.

1.12.1 10.1 Tensor shape

PYTHON IN [129]

```
# Shapes need to be in the right way
tensor_A = torch.tensor([[1, 2],
                        [3, 4],
                        [5, 6]], dtype=torch.float32)

tensor_B = torch.tensor([[7, 10],
                        [8, 11],
                        [9, 12]], dtype=torch.float32)

torch.matmul(tensor_A, tensor_B) # (this will error)
```

PYTHON IN [130]

```
# View tensor_A and tensor_B
print(tensor_A)
print(tensor_B)
```

OUTPUT

```
tensor([[1., 2.],
        [3., 4.],
        [5., 6.]])
tensor([[ 7., 10.],
        [ 8., 11.],
        [ 9., 12.]])
```

PYTHON IN [131]

```
# View tensor_A and tensor_B.T
print(tensor_A)
print(tensor_B.T)
```

OUTPUT

```
tensor([[1., 2.],
        [3., 4.],
        [5., 6.]])
tensor([[ 7.,  8.,  9.],
        [10., 11., 12.]])
```

PYTHON IN [132]

```
# The operation works when tensor_B is transposed
print(f"Original shapes: tensor_A = {tensor_A.shape}, tensor_B = {tensor_B.shape}\n")
print(f"New shapes: tensor_A = {tensor_A.shape} (same as above), tensor_B.T = {tensor_B.T.shape}\n")
print(f"Multiplying: {tensor_A.shape} * {tensor_B.T.shape} <- inner dimensions match\n")
print("Output:\n")
output = torch.matmul(tensor_A, tensor_B.T)
print(output)
print(f"\nOutput shape: {output.shape}")
```

OUTPUT

```
Original shapes: tensor_A = torch.Size([3, 2]), tensor_B = torch.Size([3, 2])

New shapes: tensor_A = torch.Size([3, 2]) (same as above), tensor_B.T = torch.Size([2, 3])

Multiplying: torch.Size([3, 2]) * torch.Size([2, 3]) <- inner dimensions match

Output:
```

```
tensor([[ 27.,  30.,  33.],
        [ 61.,  68.,  75.],
        [ 95., 106., 117.]])

Output shape: torch.Size([3, 3])
```

```
# torch.mm is a shortcut for matmul
torch.mm(tensor_A, tensor_B.T)
```

PYTHON IN [133]

```
OUTPUT
tensor([[ 27.,  30.,  33.],
        [ 61.,  68.,  75.],
        [ 95., 106., 117.]])
```

1.12.2 10.2 Tensor datatype

As mentioned, a common issue with deep learning operations is having your tensors in different datatypes.

If one tensor is in `torch.float64` and another is in `torch.float32`, you might run into some errors.

```
tensor1 = torch.arange(10., 100., 10., dtype=torch.float16)
tensor2 = torch.arange(10., 100., 10.)

tensor1@tensor2
```

PYTHON IN [136]

You can change the datatypes of tensors using `torch.Tensor.type(dtype=None)` where the `dtype` parameter is the datatype you'd like to use.

First we'll create a tensor and check its datatype (the default is `torch.float32`).

```
# Create a tensor and check its datatype
tensor = torch.arange(10., 100., 10.)
tensor.dtype
```

PYTHON IN [138]

```
OUTPUT
torch.float32
```

Now we'll create another tensor the same as before but change its datatype to `torch.float16`.

```
# Create a float16 tensor
tensor_float16 = tensor.type(torch.float16)
tensor_float16
```

PYTHON IN [140]

```
OUTPUT
tensor([10., 20., 30., 40., 50., 60., 70., 80., 90.], dtype=torch.float16)
```

And we can do something similar to make a `torch.int8` tensor.

```
tensor1@tensor2.type(torch.float16)
```

PYTHON IN [142]

```
OUTPUT
tensor(28496., dtype=torch.float16)
```

```
tensor1.type(torch.float32)@tensor2
```

PYTHON IN [143]

OUTPUT

tensor(28500.)

PYTHON IN [144]

```
# Create an int8 tensor
tensor_int8 = tensor.type(torch.int8)
tensor_int8
```

OUTPUT

tensor([10, 20, 30, 40, 50, 60, 70, 80, 90], dtype=torch.int8)

1.12.3 10.3 Tensor device

PYTHON IN [146]

```
tensor1 = torch.arange(10., 100., 10., device='cpu')
tensor2 = torch.arange(10., 100., 10., device='cuda')

tensor1@tensor2
```

PYTHON IN [147]

```
tensor1.to('cuda')@tensor2
```

OUTPUT

tensor(28500., device='cuda:0')

1.13 11. Summary and Debugging Guidelines

1.13.1 11.1 Common Error Categories in PyTorch

When developing deep learning applications, most errors fall into three primary categories. Remember this diagnostic song for systematic debugging:

“What, What, Where” - A PyTorch Debugging Mantra

“What shape are my tensors, what datatype are they, and where are they stored? What shape, what datatype, what what where!”

1.13.1.1 11.1.1 Shape Mismatches

- **Symptom:** RuntimeError involving tensor dimensions
- **Common causes:** Incompatible matrix multiplication dimensions, CNN input/output mismatches
- **Solution strategy:** Print tensor shapes before operations, use `tensor.view()` or `tensor.reshape()`

1.13.1.2 11.1.2 Datatype Incompatibilities

- **Symptom:** Operations between different precision tensors (e.g., float32 vs float64)
- **Common causes:** Mixed precision in model parameters and data
- **Solution strategy:** Use `tensor.type(torch.float32)` for consistent datatypes

1.13.1.3 11.1.3 Device Mismatches

- **Symptom:** Attempting operations between CPU and GPU tensors
- **Common causes:** Forgetting to move tensors to the same device
- **Solution strategy:** Implement device-agnostic code with `tensor.to(device)`

1.13.2 11.2 Best Practices for Production Code

1. **Device Agnostic Development:** Always write code that works on both CPU and GPU
2. **Memory Management:** Monitor GPU memory usage and implement proper cleanup
3. **Reproducibility:** Set random seeds for consistent experimental results
4. **Error Handling:** Implement robust error checking for tensor operations